A D PATEL INSTITUTE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

TRAFFIC SHORTEST PATH FINDER USING DIJKSTRA'S ALGORITHM

A Mini Project Report

A report submitted

In partial fulfilment of the requirements for the Degree of

Bachelor of Technology

In

Information Technology

Semester – V

Subject:

Design and Analysis of Algorithms (202045601)

By:

Desai Aryan M.

(12402080601009)

Faculty Guide

Minakashi Patil

Assistant Professor

Academic Year 2026–2027

Department of Information Technology

A D Patel Institute of Technology

New Vallabh Vidyanagar, Vitthal Udyognagar – 388121

Anand, Gujarat, India

CERTIFICATE

This is to certify that the mini project work embodied in this report entitled “Traffic Shortest Path Finder Using Dijkstra’s Algorithm” was carried out by **Desai Aryan M. (12402080601009)** at A D Patel Institute of Technology in partial fulfilment of the requirements for the Bachelor of Technology degree in Information Technology. This project work has been carried out under the guidance and supervision of the faculty guide and is to the satisfaction of the department.

Date: _____

Place: New Vallabh Vidyanagar

Lab Teacher	Head of Department
Minakashi Patil	Dr. Priyang Bhatt
Assistant Professor	Associate Professor & Head

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to A D Patel Institute of Technology and the Department of Information Technology for providing the opportunity to undertake this mini project, “Traffic Shortest Path Finder Using Dijkstra’s Algorithm,” as part of the Design and Analysis of Algorithms course.

I am thankful to my faculty guide, Minakashi Patil (Assistant Professor), for the guidance and suggestions that supported the planning and preparation of this project. The project gave me an opportunity to apply a classical greedy algorithm to a practical urban-routing problem and to understand how implementation choices affect performance.

I also appreciate the support of my classmates and everyone who offered assistance or encouragement during the project. Finally, I am grateful to my family for their continued support.

Desai Aryan M.
(12402080601009)

TABLE OF CONTENTS

CERTIFICATE . . . 2
ACKNOWLEDGEMENT . . . 3
TABLE OF CONTENTS . . . 4
LIST OF TABLES . . . 5
ABSTRACT . . . 6
CHAPTER 1: INTRODUCTION . . . 7
1.1 Background . . . 7
1.2 Problem Statement . . . 7
1.3 Objectives . . . 7
1.4 Scope . . . 8
1.5 Existing and Proposed System . . . 8
1.6 Applications . . . 8
CHAPTER 2: THEORETICAL BACKGROUND . . . 9
2.1 The Shortest-Path Problem . . . 9
2.2 Greedy Strategy . . . 9
2.3 Dijkstra's Algorithm . . . 9
2.4 Algorithm Steps . . . 10
2.5 Data Structures Used . . . 10
2.6 Complexity Analysis . . . 10
CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY . . . 11
3.1 System Overview . . . 11
3.2 Road Network Model . . . 11
3.3 Working of the System . . . 12
3.4 Algorithms Used . . . 13
3.5 Complexity Analysis . . . 13
CHAPTER 4: IMPLEMENTATION AND TESTING . . . 14
4.1 Technologies Used . . . 14
4.2 Implementation Details . . . 14
4.3 Test Cases and Results . . . 15
4.4 Results and Discussion . . . 16
4.5 Limitations . . . 17
CHAPTER 5: CONCLUSION AND FUTURE SCOPE . . . 18
5.1 Conclusion . . . 18
5.2 Future Scope . . . 18
REFERENCES . . . 19

LIST OF TABLES

Table 1. Functional requirements of the Traffic Shortest Path Finder

Table 2. Comparison of shortest-path algorithms

Table 3. Road network: nodes and roads

Table 4. Complexity analysis of operations

Table 5. Technologies used in the project

Table 6. Test cases and results

ABSTRACT

Finding a fast route through a city is a routine requirement, yet the geographically shortest road is not always the quickest, because congestion can make a short road slow. This mini project presents the Traffic Shortest Path Finder, an application that models a city as a weighted graph and uses Dijkstra's Algorithm to find the least-cost route between two locations while accounting for traffic.

The road network consists of 8 locations (nodes A to H, such as Central Terminus, City Center and Airport Terminal) joined by 14 bidirectional roads with lengths in kilometres. Each road can be switched between normal traffic (cost multiplier 1.0, assumed speed 50 km/h) and heavy traffic (cost multiplier 2.5, assumed speed 20 km/h). Dijkstra's Algorithm runs on these effective costs, so toggling congestion on a road can change the recommended route.

The core algorithm is implemented in C (C99) as a menu-driven console program using an adjacency cost matrix and a linear-scan selection of the minimum-distance node. An equivalent Python terminal runner is provided, along with an interactive browser version built with HTML, CSS and JavaScript that draws the network as SVG and highlights the chosen route. For each query the program prints a step-by-step execution trace, the route, the physical distance, the effective cost and the estimated travel time.

The implemented version runs in $O(V^2)$ time and $O(V^2)$ space (the matrix); a binary-heap variant on an adjacency list would give $O((V + E) \log V)$. In the main demonstration, the route from A to H is A → D → H (17 km, 20.4 minutes) on clear roads. When the roads A–D and D–H are congested, the program reroutes to A → C → F → H (18 km, 21.6 minutes). The network is fixed in the source code and the program does not read real traffic data.

CHAPTER 1: INTRODUCTION

1.1 Background

Urban road networks carry large volumes of traffic, and travel time depends on both road length and current congestion. Navigation tools must therefore choose routes using a cost that reflects travel conditions, not distance alone.

From an algorithmic perspective, route finding is a standard shortest-path problem on a weighted graph. Intersections or landmarks become vertices, roads become edges, and the cost of an edge represents the effort of using that road. Dijkstra's Algorithm is a well-known greedy solution for graphs whose edge weights are non-negative, which is the case for road lengths and congestion multipliers.

1.2 Problem Statement

Given a road network represented as a graph G = (V, E), where V is a set of city locations and E is a set of bidirectional roads with physical length d(u, v) in kilometres, and where each road has a traffic multiplier T(u, v) (1.0 for normal traffic, 2.5 for heavy traffic), the task is to find the route from a source S to a destination D that minimizes the total effective cost, and to recompute the route when the traffic state of a road changes.

The effective cost of a road is: **effective weight = d × T**.

1.3 Objectives

- To model a city road network as a weighted, undirected graph.
- To implement Dijkstra's Algorithm from scratch in C, without graph libraries.
- To represent traffic congestion through an effective-weight multiplier and show how it changes the optimal route.
- To display a step-by-step execution trace so that the working of the greedy algorithm can be followed.
- To report distance, effective cost and estimated travel time for the chosen route.
- To provide an interactive visual version in the browser for demonstration.
- To analyze the time and space complexity of the implementation.

1.4 Scope

The project covers a fixed 8-node, 14-road network defined in the source code. It supports choosing any source and destination, toggling traffic on any road, running preset demonstrations, and viewing the road table. It is an educational project intended to demonstrate DAA concepts and is not a complete navigation product.

1.5 Existing and Proposed System

In a static approach, the route is chosen using road lengths only. Such a route stays the same even if one of its roads becomes congested, so the result may no longer be the fastest. Choosing a route by inspecting a map manually also becomes error-prone as the number of roads grows.

The proposed system computes the route automatically with Dijkstra's Algorithm and recomputes it whenever traffic conditions are changed. It also shows the algorithm's trace, which makes the greedy selection and edge relaxation visible.

Table 1. Functional requirements of the Traffic Shortest Path Finder

Requirement type	Requirement
Functional	Find the least-cost route between any two of the 8 locations.
Functional	Toggle any road between normal and heavy traffic.
Functional	Show the execution trace, route, distance, effective cost and travel time.
Functional	Provide preset demonstrations for normal flow and congestion rerouting.
Functional	Reject identical start/destination and invalid location or menu input.
Constraint	The road network and traffic state are fixed in the program and not saved between runs.

1.6 Applications

- Demonstrating the greedy strategy and edge relaxation in Dijkstra's Algorithm.
- Understanding how edge-weight changes alter shortest paths.
- Teaching graph representation using adjacency matrices and adjacency lists.
- Serving as a starting point for route-planning tools, delivery routing and network routing studies.

CHAPTER 2: THEORETICAL BACKGROUND

2.1 The Shortest-Path Problem

The single-source shortest-path problem asks for the minimum-cost path from one vertex to all others (or to a chosen destination) in a weighted graph. The cost of a path is the sum of its edge weights. In a road network the edges are undirected, so each road is stored in both directions.

2.2 Greedy Strategy

A greedy algorithm builds a solution by repeatedly making the locally best choice. Dijkstra's Algorithm keeps a set of settled vertices whose shortest distance is final. At each step it selects the unsettled vertex with the smallest tentative distance and settles it. Because all edge weights are non-negative, no later path can reach that vertex more cheaply, so the choice is safe and the final result is globally optimal.

2.3 Dijkstra's Algorithm

For every vertex, the algorithm maintains a tentative distance dist[v], a parent pointer parent[v] and a visited flag. The source starts with distance 0 and all others with infinity. The key operation is **relaxation**: when a vertex u is settled, each unsettled neighbour v is checked, and if dist[u] + w(u, v) is smaller than dist[v], then dist[v] and parent[v] are updated.

The algorithm requires non-negative edge weights. With negative weights, a settled vertex could later be improved, which breaks the greedy invariant; the Bellman-Ford algorithm should be used in that case.

Table 2. Comparison of shortest-path algorithms

Algorithm	Graph type	Time complexity	Use in this project
Breadth-First Search	Unweighted	O(V + E)	Not suitable; roads have different costs.
Dijkstra (array / matrix)	Non-negative weights	$O(V^2)$	**Implemented** in C, Python and JavaScript.
Dijkstra (binary heap, adjacency list)	Non-negative weights	O((V + E) log V)	Not implemented; suggested for large networks.
Bellman-Ford	Weights may be negative	O(V × E)	Not needed; no negative weights.
Floyd-Warshall	All pairs	$O(V^3)$	Not needed; one source and one destination are queried.

2.4 Algorithm Steps

Pseudocode of Dijkstra's Algorithm as implemented:

```
1. dist[v] = INF, parent[v] = -1, visited[v] = false for all v
2. dist[source] = 0
3. repeat up to V times:
     u = unvisited vertex with the smallest dist[u]
     if no such vertex or dist[u] == INF: stop
     visited[u] = true
     if u == destination: stop
     for each unvisited neighbour v of u:
         if dist[u] + cost(u, v) < dist[v]:
             dist[v] = dist[u] + cost(u, v)
             parent[v] = u
4. if dist[destination] == INF: destination is unreachable
5. rebuild the route by following parent[] from destination to source
```

2.5 Data Structures Used

- **Road structure:** stores an id, the two end nodes, the distance and a heavy-traffic flag.
- **roads[14] array:** the list of all roads in the network.
- **Cost matrix costMatrix[V][V]:** built for each query from effective costs; INF marks no direct road.
- **dist[], parent[], visited[] arrays:** the working state of Dijkstra's Algorithm.
- **path[] array:** the route rebuilt from parent pointers.
- **Adjacency list (JavaScript version):** each node holds its neighbours with distance and effective weight.

2.6 Complexity Analysis

Let V be the number of vertices and E the number of edges. The implementation selects the minimum-distance vertex by scanning all V vertices, repeated up to V times, which costs $O(V^2)$. Relaxation examines each row of the matrix once, which is also $O(V^2)$ in total. Building the matrix takes $O(V^2)$ to initialize and O(E) to fill. Hence the C implementation runs in **$O(V^2)$** time and uses **$O(V^2)$** space for the matrix plus O(V) for the working arrays.

With an adjacency list and a binary min-heap, the running time improves to $O((V + E) \log V)$ and the space to O(V + E), which matters for large sparse road networks.

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

3.1 System Overview

The system is a menu-driven C console program. Its main modules are the road network definition, the effective-cost function, Dijkstra's Algorithm with trace output, the road table and traffic toggle, the preset demonstrations and the main menu loop. The same logic is also provided as a Python terminal runner and as a browser application (index.html, style.css, script.js) that draws the network and highlights the route.

3.2 Road Network Model

The network has V = 8 vertices and E = 14 bidirectional edges.

Table 3. Road network: nodes and roads

Node	Location
A	Central Terminus
B	North Tech Hub
C	South Market
D	City Center
E	Industrial Zone
F	Metro Plaza
G	University & Medical Hub
H	Airport Terminal

Road ID	Link	Distance (km)
1	A – B	5
2	A – C	6
3	A – D	8
4	B – E	4
5	B – D	4
6	C – D	4
7	C – F	4
8	D – E	3
9	D – F	3
10	E – G	4
11	D – G	6
12	F – H	8
13	G – H	5
14	D – H	9

A heavy-traffic road has effective cost = distance × 2.5 and is assumed to be travelled at 20 km/h; a normal road has effective cost = distance and speed 50 km/h. Travel time is the sum of distance ÷ speed over the roads of the chosen route.

3.3 Working of the System

1 Start the program with all roads in the normal traffic state.

2 Display the menu: find a route, toggle road traffic, preset demo 1, preset demo 2, view the road table, exit.

3 Read the menu choice; for an invalid number show a message and show the menu again.

4 For **Find Shortest Route**, read the start and destination letters (case-insensitive), validate them and run Dijkstra's Algorithm.

5 For **Toggle Road Traffic**, show the road table, read a road ID (1 to 14) and flip its heavy-traffic flag.

6 For the **preset demos**, set a fixed traffic state and run the search from A to H.

7 Build the cost matrix from the current effective costs, run the algorithm, print the trace table, rebuild the route from the parent pointers and print the route breakdown.

8 Repeat until Exit is chosen.

3.4 Algorithms Used

Dijkstra's Algorithm. Implemented as described in Section 2.4. The trace table lists, for each step, the settled node and the neighbours whose distances were improved. The search stops as soon as the destination is settled, which avoids unnecessary work.

Route reconstruction. Starting at the destination, the program follows parent[] back to the source and then reads the path in reverse order.

Distance and time computation. For each pair of consecutive nodes on the route, the matching road is found in roads[] to add its physical distance and its travel time at 50 or 20 km/h.

3.5 Complexity Analysis

Table 4. Complexity analysis of operations

Operation	Time	Space
Build cost matrix	$O(V^2) + O(E)$	$O(V^2)$
Dijkstra main loop (selection + relaxation)	$O(V^2)$	$O(V)$
Route reconstruction	$O(V)$	$O(V)$
Distance and time computation	$O(V \times E)$	$O(1)$
Toggle traffic	$O(1)$	$O(1)$

The distance and time step looks up each path edge by scanning the road list; since a path has at most $V - 1$ edges, this is $O(V \times E)$, which is $O(V^2)$ for road networks where E is proportional to V. Overall the query cost is $O(V^2)$ in the dominant case. For $V = 8$ and $E = 14$ the running time is negligible.

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Technologies Used

Table 5. Technologies used in the project

Technology	Purpose
C (C99), dijkstra.c	Main menu-driven implementation of Dijkstra's Algorithm with trace output.
Python 3, terminal_runner.py	Equivalent terminal version, used as a fallback if no C compiler is available.
HTML5, CSS3	Structure and styling of the browser version.
JavaScript (ES6), script.js	Dijkstra's Algorithm with an adjacency list, plus interactive controls.
SVG	Drawing the road network and highlighting the shortest route.
GCC	Compiling the C program (gcc dijkstra.c -o dijkstra).

No external libraries, database or server are needed.

4.2 Implementation Details

Data definition. Nodes are numbered 0 to 7 (A to H). Each road is a Road structure with id, u, v, distance and isHeavy. getEffectiveCost() returns distance × 2.5 for a heavy road and distance otherwise.

dijkstra(src, dest). Rejects identical source and destination, builds a symmetric cost matrix from effective costs, initializes dist, parent and visited, and runs the greedy loop. INF is defined as 999999.0. The loop prints one trace row per settled node and stops when the destination is settled or no reachable unvisited node remains. If dist[dest] is still INF, the destination is reported as unreachable.

Traffic handling. displayNetwork() prints the road table with status and effective cost. toggleTraffic() flips the flag of the chosen road. The result display reports the physical distance, the effective cost, the estimated travel time and whether the chosen route contains congested roads.

Preset demos. Demo 1 clears all congestion and runs A → H. Demo 2 marks A–D (road 3) and D–H (road 14) as heavy and runs A → H.

Input handling. Location letters are converted to uppercase with toupper() and range-checked; the menu rejects numbers outside 1 to 6 with a message.

4.3 Test Cases and Results

The C program was compiled with gcc using -Wall (no warnings were reported) and each case below was executed; the results are taken from the actual program output.

Table 6. Test cases and results

ID	Test input / action	Expected result	Actual result	Status
TC-01	Demo 1: A to H, all roads clear	Least-cost route found	A → D → H; 17.0 km; cost 17.0; 20.4 min	Pass
TC-02	Demo 2: A–D and D–H heavy, A to H	Route avoids congested roads	A → C → F → H; 18.0 km; cost 18.0; 21.6 min	Pass
TC-03	Custom route B to G, all clear	Least-cost route found	B → E → G; 8.0 km; cost 8.0; 9.6 min	Pass
TC-04	Lowercase input “b” and “g”	Same as uppercase	Accepted; same result as TC-03	Pass
TC-05	Road 1 (A–B) heavy, then A to B	Cheaper detour preferred over the heavy direct road	A → D → B; 12.0 km; cost 12.0 (direct road costs 12.5)	Pass
TC-06	Roads 1 (A–B) and 3 (A–D) heavy, A to H	South route chosen	A → C → F → H; 18.0 km; 21.6 min	Pass
TC-07	Start and destination both A	Error message	“Starting point and destination cannot be identical.”	Pass
TC-08	Invalid location letter (Z)	Error message	“Invalid locations entered. Please choose letters between A and H.”	Pass
TC-09	Menu choice 9	Invalid-choice message	“Invalid choice. Please enter a number from 1 to 6.”	Pass
TC-10	View road table (option 5)	14 roads with status and cost	14 roads listed, all “Normal (1.0x)” initially	Pass

Trace of Demo 1 (A to H, clear roads)

Step	Settled node	Distances improved
1	A	B:5, C:6, D:8
2	B	E:9
3	C	F:10
4	D	G:14, H:17
5	E	G:13
6	F	no updates
7	G	no updates
8	H	destination settled

The trace shows the greedy rule at work: node D is settled with distance 8, and its edge to H gives the final distance of 17, which is not improved by the later nodes.

4.4 Results and Discussion

On clear roads the program selects A → D → H at 17 km. When A–D and D–H become heavy, their effective costs rise to 20 and 22.5, and the program reroutes through A → C → F → H at 18 km. The new route is physically one kilometre longer but avoids the congested roads, which shows why a cost that includes traffic gives better routes than distance alone.

TC-05 shows a second effect: with the direct road A–B heavy (effective cost 12.5), the cheaper way to reach B is the detour A → D → B (cost 12.0), even though it is 7 km longer than the 5 km direct road.

A point worth noting in Demo 2 is a **tie**. The north route A → B → E → G → H has effective cost 5 + 4 + 4 + 5 = 18, the same as the south route A → C → F → H (6 + 4 + 8 = 18). Because relaxation updates a distance only when the new value is strictly smaller, the first route that reached H with cost 18 is kept, which is the south route. Both routes are equally optimal; the on-screen message of the preset mentions the north route, but the program's reported route is the south one.

The implemented algorithm uses $O(V^2)$ selection, which is simple and efficient for 8 nodes. For large sparse road networks, an adjacency list with a binary heap would be the better choice.

4.5 Limitations

- The road network is hard-coded; there is no way to add or edit nodes and roads at run time.
- Traffic is a manual two-state toggle (1.0× or 2.5×), not live data.
- Speeds (50 and 20 km/h) are fixed assumptions used for the time estimate.
- Only one route is returned; ties between equal-cost routes are resolved by the order of evaluation.
- The implemented minimum selection is a linear scan ($O(V^2)$), not a heap.
- Results are not saved between runs.

CHAPTER 5: CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The Traffic Shortest Path Finder shows how Dijkstra's Algorithm can be applied to a practical routing problem. The C program models an 8-node, 14-road city network, lets the user introduce congestion on any road, and recomputes the best route using effective costs. The step-by-step trace makes the greedy choice and edge relaxation easy to follow, and the browser version presents the same logic visually.

The implemented version runs in $O(V^2)$ time and $O(V^2)$ space, and the analysis shows how an adjacency list with a heap would reach $O((V + E) \log V)$ for larger networks. All test cases executed gave the expected results, including the rerouting demonstration under congestion.

5.2 Future Scope

- Replace the linear-scan selection with a binary min-heap and use adjacency lists for large networks.
- Load the road network and distances from a file instead of hard-coding them.
- Read live or time-of-day traffic data to set the congestion multipliers automatically.
- Support multiple congestion levels instead of a single heavy state.
- Report alternative routes, and show all tied optimal routes.
- Support one-way roads by using a directed graph.
- Compare against A* search to speed up point-to-point queries.
- Add automated tests and measure running time for larger generated networks.

REFERENCES

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. Introduction to Algorithms. MIT Press.

2. Sedgewick, R., and Wayne, K. Algorithms. Addison-Wesley.

3. Kernighan, B. W., and Ritchie, D. M. The C Programming Language. Prentice Hall.

4. Dijkstra, E. W. “A note on two problems in connexion with graphs.” Numerische Mathematik, 1(1), 1959.

5. Project source files: dijkstra.c, terminal_runner.py, index.html, script.js, style.css, README.md.